SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

CSA1505 – CLOUD COMPUTING AND BIG DATA ANALYTICS

ASSIGNMENT 1 – CO1

Design, Analyse and Validate a Cloud-Based Solution for Migrating a Legacy University Management System

CO1: Apply cloud computing technologies including hardware–software evolution, virtualization, web services, and service models to develop cloud-based solutions.

Field	Details	Field	Details
Student Name	G Rahul	Register Number	192572008
Department / Section	AI & ML	Academic Year / Semester	2nd
Faculty Name	Dr. Joseph Antony Rajan	Nature of Assignment	Individual
Duration	1 Week	Total Marks	50
Submission Date		Evaluation Date	

MARK SHEET (Rubric-based evaluation)

Task	Title	Max	Awarded	Evidence / evaluation focus
1	Introduce Cloud Computing	3		Definition; characteristics; deployment models; scenario mapping
2	Trace Evolution and Hardware Evolution	3		Timeline; hardware evolution; relevance to university
3	Analyse Internet Software Evolution	3		Web/software evolution; SOA; APIs; scenario relevance
4	Apply Server Virtualization	5		Hypervisor types; VM/container comparison; consolidation; limitations
5	Apply Web Services	4		SOA and SOAP/REST; three university APIs; request/response examples
6	Analyse IaaS, PaaS and SaaS	5		Responsibility layers; five-application mapping; comparison; providers
7	Examine XaaS	3		Four or more variants; two justified choices; provider risks

8	Design Cloud Solution Architecture	5		Labelled diagram; deployment/service models; scaling; security; DR
9	Hands-on Implementation and Documentation	4		Running VM/container; PaaS/SaaS; tests; actual screenshots
10	Evaluate and Recommend Final Solution	7		Three-way comparison; 3-year costs; risks; phased roadmap
11	Day-4 Design Challenge	8		Impact; evaluation; adaptation; evidence-based defence
	TOTAL	**50**		**Percentage :**

EVIDENCE VERIFICATION CHECKLIST

Tasks	**Evidence expected**	**Status (✓ / P / ✗)**	**Evaluator note**
1–3	Conceptual explanations, comparisons and relevant illustrations	✓	Explanations and relevant scenario mapping documented
4	Virtualization calculations with stated assumptions and peak-load caution	✓	Calculations present; practical capacity validation recommended
5–7	API examples, service responsibility table and justified XaaS choices	✓	Required conceptual evidence documented
8	Labelled architecture diagram, caption below figure and component justification	✓	Architecture documented; sizing assumptions need testing
9	Actual code, environment details, execution logs, API outputs and screenshots	P	Procedures provided; actual execution evidence not established
10	Cost assumptions, 3-year totals, risk controls and migration roadmap	✓	Estimates and rationale documented
11	Day-4 changes, 10× load strategy, 99.9% target and 40-college isolation	✓	Design analysis documented; practical validation pending

IMPLEMENTATION EVIDENCE CHAIN

Evidence item	Status	**Observation**
Source code	✓	Flask REST API and Dockerfile provided in the proposed implementation
Virtualization	P	Docker container configuration documented; running-container evidence required
PaaS or SaaS	P	Managed web-service deployment procedure described; live deployment not verified
Functional test	P	Expected HTTP 200 response and JSON output documented; actual output required
Failure case	P	HTTP 404 test case documented; actual execution record required
Screenshots	P	Screenshot requirements identified; authentic screenshots need verification
Reproducibility	✓	Build commands, run commands, endpoints and expected results specified

G Rahul	_____	_____

Student Signature	Faculty Signature	Program Director

OVERALL REMARK

MANDATORY SUBMISSION EVIDENCE CHECK

✓ = Present in documented answer; P = Partly present; ✗ = Missing / not satisfied

Task	Evidence expected	Status	Evaluator note
1	Cloud definition, characteristics, deployment models and scenario mapping	✓	All conceptual requirements covered
2	Evolution timeline, hardware development and university relevance	✓	Well explained with relevant examples
3	Internet software evolution, SOA and APIs	✓	Clear explanation and scenario relevance
4	Hypervisors, VM/container comparison and consolidation calculations	✓	Calculations included; peak-load limitations discussed
5	SOAP/REST comparison and three university APIs	✓	API methods and request/response examples documented
6	IaaS/PaaS/SaaS responsibility layers and application mapping	✓	All five applications categorized with justification
7	XaaS variants and two justified selections	✓	Six variants explained; DBaaS and DRaaS recommended
8	Labelled architecture, scaling, security and disaster recovery	✓	Design and component justifications documented
9	Working container, PaaS/SaaS deployment, actual tests and screenshots	P	Code and procedures provided; actual execution evidence requires verification
10	Three-year cost comparison, risks and migration roadmap	✓	Cost assumptions, calculations and recommendations included
11	Impact analysis, 10× load, 99.9% availability and tenant isolation	✓	Changed requirements evaluated; load-test evidence recommended

IMPLEMENTATION EVIDENCE CHAIN

Evidence item	Status	Observation
Source code	✓	Python Flask REST API and Dockerfile documented
Virtualization	P	Docker build and run procedures provided; execution screenshot required
PaaS or SaaS	P	Cloud deployment steps described; actual deployment confirmation required

Functional test	P	HTTP 200 and expected JSON response documented; actual test output required
Failure case	P	HTTP 404 test case included; execution evidence required
Screenshots	P	Required screenshots identified but authenticity and completeness not verified
Reproducibility	✓	Installation, configuration, commands and testing steps documented
Interpretation	✓	Technical outcomes and implementation limitations explained
Code and report consistency	✓	Proposed API, Docker configuration and testing procedure are aligned

SUBMISSION CHECKLIST (Not Separately Marked)

Requirement	**Status**
All 11 tasks answered	☒ Yes ☐ Partly ☐ No
Cloud concepts and comparisons explained	☒ Yes ☐ Partly ☐ No
Virtualization calculations included	☒ Yes ☐ Partly ☐ No
API examples and service models explained	☒ Yes ☐ Partly ☐ No
Cloud architecture diagram included	☒ Yes ☐ Partly ☐ No
Implementation and testing evidence provided	☐ Yes ☒ Partly ☐ No
Three-year cost analysis and roadmap included	☒ Yes ☐ Partly ☐ No
Day-4 design challenge completed	☒ Yes ☐ Partly ☐ No
Figures and screenshots included	☐ Yes ☒ Partly ☐ No
Table and figure numbering correct	☐ Yes ☒ Partly ☐ No
Viva recommended	☒ Yes ☐ No

CSA15 – Cloud Computing and Big Data Analytics

ASSIGNMENT 1

CO1: *Apply cloud computing technologies including hardware–software evolution, virtualization, web services, and service models to develop cloud-based solutions.*

Assignment Title: Design, Analyse and Validate a Cloud-Based Solution for Migrating a Legacy University Management System (Total Marks – 50)

Duration: 1 week

Scenario – Smart Campus Cloud Migration

A multi-campus university currently runs its Student Information System, Learning Management System, Library System, Exam Portal and Email on ageing physical servers in a single on-premise data centre. Each application has its own dedicated server, which stays idle for most of the year but fails to cope during admissions, result publication and online-exam weeks.

The university's current environment is:

- 12 physical servers, average CPU utilisation of about 15%, with peaks above 95% during admissions and exams
- Approximately 25,000 students and 2,000 staff across 4 campuses
- Applications built as large monolithic programs, with no public interfaces for mobile apps or partner colleges
- High capital expenditure, slow procurement (6–8 weeks per server) and no disaster recovery site

As a cloud consultant, you must study how computing evolved to the cloud model, and then design a cloud-based solution that uses virtualization, web services and an appropriate mix of service models (IaaS, PaaS, SaaS and XaaS) for the university. All recommendations must be justified in terms of cost, scalability, availability and manageability.

Design Challenge:

On Day 4, the university announces that it will launch a mobile app and open student-record and fee-payment services to 40 affiliated colleges. Expected load is 10 times the current peak, with a requirement of 99.9% availability and a limited operating budget. Re-evaluate your original design and decide whether it should be retained, modified or redesigned.

TASK 1 – Introduce Cloud Computing and Map It to the Scenario (3 Marks)

Design and Propose cloud computing model for the context of the university's problems.

- Definition of cloud computing and the five essential characteristics (on-demand self-service, broad network access, resource pooling, rapid elasticity, measured service)
- Deployment models (public, private, hybrid, community) and which one suits the university
- Three problems of the current on-premise setup and how each cloud characteristic addresses them
- Benefits and limitations of cloud adoption for the university

Evidence Expected

A clear explanation of cloud characteristics and deployment models, mapped to the university's actual problems.

TASK 2 – Trace the Evolution and Hardware Evolution (3 Marks)

Trace how computing infrastructure evolved to make cloud computing possible.

- A timeline from mainframes, minicomputers and PCs, to client–server, clusters, grid computing, utility computing and cloud
- Hardware evolution: multi-core processors, commodity servers, high-capacity storage, high-speed networking and data centres
- How each stage solved a limitation of the previous one
- Which stage the university's current system corresponds to, with reasons

Evidence Expected

A labelled timeline or diagram, with a justified explanation of how hardware advances enabled the cloud.

TASK 3 – Analyse Internet Software Evolution (3 Marks)

Explain how internet software evolved and why it supports cloud services.

- Evolution from static web pages to dynamic web, Web 2.0, SOA, mashups and cloud-hosted applications
- Role of browsers, HTTP, AJAX and APIs in delivering software over the internet
- Comparison between desktop software, client–server software and web-delivered software
- How this evolution affects the design of the university's applications

Evidence Expected

A comparative explanation of software evolution stages, linked to the university's application design.

TASK 4 – Apply Server Virtualization (5 Marks)

Apply server virtualization to consolidate the university's 12 physical servers.

- Explain how a hypervisor works, and distinguish Type-1 and Type-2 hypervisors, with examples
- Compare full virtualization, para-virtualization, OS-level virtualization (containers) and hardware-assisted virtualization
- Propose which applications should run as virtual machines and which as containers, with reasons
- Calculate the consolidation: with an average utilisation of 15% per server and a target host utilisation of 70%, estimate how many physical hosts are needed (show your assumptions and working)
- Discuss live migration, snapshots and resource over-commitment as benefits, and the risks of virtualization (noisy neighbour, hypervisor failure, security isolation)

Show your calculation. A final number without working will not receive full credit.

Evidence Expected

A justified virtualization plan with a worked consolidation calculation and discussion of benefits and risks.

TASK 5 – Apply Web Services (4 Marks)

Analyse web services to expose the university's functions to other systems.

- Overview of web services and their role in cloud and SOA
- Compare SOAP, WSDL, UDDI and RESTful services (HTTP methods, JSON/XML)

- Identify at least three university functions to expose as web services (for example, result lookup, fee payment, library search) and define the endpoint, method and sample request/response for each
- Justify whether SOAP or REST is more suitable for each function

Evidence Expected

A defined set of web services with sample request/response formats and a justified choice of SOAP or REST

TASK 6 – Analyse the Service Models: IaaS, PaaS and SaaS (5 Marks)

Analyse which service model fits each university application.

- Define IaaS, PaaS and SaaS, and show who manages which layer (hardware to application) in each model
- Categorise the five university applications (SIS, LMS, Library, Exam Portal, Email) into IaaS, PaaS or SaaS, with reasons
- Compare the models on control, cost, scalability, maintenance effort and vendor lock-in
- Give one real provider example for each model

Present your **categorisation in a table**, but explain the **reasoning behind** each **decision**. A table alone is not sufficient.

Evidence Expected

A reasoned mapping of applications to service models, supported by a layer-wise responsibility comparison.

TASK 7 – Examine XaaS (3 Marks)

Examine the broader XaaS (Anything-as-a-Service) concept for the university.

- Explain XaaS and describe at least four variants (for example DaaS, BaaS/MBaaS, FaaS, DBaaS, Security-as-a-Service, Disaster-Recovery-as-a-Service)
- Select two XaaS offerings that would benefit the university, and justify them
- Discuss the risks of over-reliance on multiple 'as-a-Service' providers

Evidence Expected

An analysis of XaaS variants, with justified selection of two offerings for the university.

TASK 8 – Design the Cloud Solution Architecture (5 Marks)

Bloom's Level: Level 6 – Create | *Action verbs: Design, Construct, Propose*

Create a complete cloud architecture for the university by combining virtualization, web services and service models.

- An architecture diagram showing users, load balancer, web/application tier, database tier, storage, API layer and DR region
- The deployment model and the service model chosen for each component
- How elasticity (auto-scaling) will handle admissions and exam peaks
- Basic security and availability measures (access control, encryption, backup, multi-zone deployment)

Evidence Expected

A labelled architecture diagram with written justification for each component and design decision.

TASK 9 – Hands-on Implementation and Documentation (4 Marks)

Implement a small proof-of-concept of your design using a free-tier cloud account or a local virtualization tool.

- Create a virtual machine or container (using AWS/Azure/GCP free tier, VirtualBox, or Docker) and host a simple web page or REST API for one university function
- Deploy a SaaS or PaaS component (for example a cloud-hosted database, a serverless function or a static site)
- Test the web service with a client (browser, Postman or curl) and record the output
- Provide screenshots, configuration steps and any issues faced

Evidence Expected

Annotated screenshots and clear, reproducible steps showing that the proof-of-concept works.

TASK 10 – Evaluate and Recommend the Final Solution (7 Marks)

Evaluate your design and recommend the final cloud solution for the university.

- Compare on-premise, IaaS-based and PaaS/SaaS-based options on cost (CapEx vs OpEx), scalability, availability, security and effort
- Estimate a simple 3-year cost comparison using stated assumptions

- Identify the main risks (vendor lock-in, data privacy, downtime, skills gap) and give mitigation steps
- Provide your final recommendation and a short migration roadmap (which application moves first, and why)

Your recommendation must be supported by the analysis in the previous tasks. Choosing a popular cloud provider without justification is not sufficient.

Evidence Expected

A technically justified recommendation, supported by a cost comparison, risk analysis and migration roadmap.

DAY-4 DESIGN CHALLENGE

TASK 11 – Re-evaluate and Adapt the Design (8 Marks)

Changed Requirement

The university will now launch a mobile app and open student-record and fee-payment services to 40 affiliated colleges. Expected load is 10 times the current peak, availability must be 99.9%, and the operating budget is limited.

Re-evaluate your original design in response to these changed conditions. You may:

RETAIN → MODIFY → or REDESIGN

your original solution.

Impact Analysis – 2 Marks

Discuss:

- What has changed from the original problem
- Which original assumptions (load, users, availability, cost) are affected
- Why the web services and APIs now become critical
- How multi-tenant access by 40 colleges affects security and isolation

Evidence Expected: A clear analysis of how the changed requirements affect the original cloud design.

Evaluation of the Original Design – 2 Marks

Discuss:

- Whether the chosen service models and virtualization plan can handle a 10x load
- Whether auto-scaling and the single-region deployment are sufficient for 99.9% availability (about 8.76 hours of downtime allowed per year)
- Whether cost remains acceptable under the new load
- Whether another service model or XaaS offering would give better overall results

Evidence Expected: A technically supported evaluation of the original design under the new conditions.

Adaptation or Redesign – 2 Marks

Discuss:

- Decide whether to retain, modify or redesign the solution
- Describe the revised architecture (for example API gateway, container orchestration, serverless components, multi-zone deployment or a hybrid model)
- Explain what changed in the service model mix and in the virtualization approach
- If you retain the original design, give sufficient technical justification for why no change is required

Evidence Expected: An appropriate design response to the changed requirement, with clear technical reasoning.

Evidence-Based Defence – 2 Marks

Discuss:

- Capacity and availability calculations
- Cost comparison between the original and the revised design
- Security and multi-tenancy considerations
- Trade-offs such as vendor lock-in, complexity and migration effort

Evidence Expected: A well-supported final decision that follows logically from the analysis.

MARK DISTRIBUTION

Task	Title	Bloom's Level	Marks
Task 1	Cloud Computing – Introduction & Characteristics	L2 – Understand	3
Task 2	Evolution & Hardware Evolution	L2 – Understand	3
Task 3	Internet Software Evolution	L2 – Understand	3
Task 4	Server Virtualization	L3 – Apply	5
Task 5	Web Services	L3 – Apply	4
Task 6	Service Models: IaaS, PaaS, SaaS	L4 – Analyze	5
Task 7	XaaS	L4 – Analyze	3
Task 8	Design the Cloud Solution Architecture	L6 – Create	5
Task 9	Hands-on Implementation & Documentation	L3 – Apply	4
Task 10	Evaluate & Recommend the Final Solution	L5 – Evaluate	7
	Subtotal		

Task 11	Day-4 Design Challenge	L4 / L5 / L6	8
	TOTAL		50

ASSIGNMENT 1 – CO1

ANSWERS

CSA15 – CLOUD COMPUTING AND BIG DATA ANALYTICS

Assignment Title: Design, Analyse and Validate a Cloud-Based Solution for Migrating a Legacy University Management System

TASK 1 – INTRODUCTION TO CLOUD COMPUTING

1.1 Definition

Cloud computing is the delivery of computing resources such as servers, storage, databases, networking, software and processing power over the internet. Users can access these resources when required and pay according to usage instead of purchasing and maintaining all hardware themselves.

1.2 Five Essential Characteristics

On-demand self-service: Users can request computing resources automatically without waiting for a service provider. Example: the university can create a test server during examination preparation.

Broad network access: Services are available over standard networks through laptops, mobiles and tablets. Example: students can check results from any campus or location.

Resource pooling: A provider shares a pool of computing resources among multiple customers while keeping their data logically separated. Example: the university's virtual machines use shared cloud infrastructure.

Rapid elasticity: Resources can increase or decrease quickly according to demand. Example: extra application servers can be added during admissions and removed after the peak.

Measured service: Resource usage is monitored and billed according to consumption. Example: the university pays for storage and compute capacity actually used.

1.3 Deployment Models

Model	Meaning	Suitability to the university
Public cloud	Infrastructure is operated by a cloud provider and shared securely by many customers.	Useful for scalable web hosting, backups and public-facing services.
Private cloud	Cloud infrastructure is dedicated to one organisation.	Can be used for highly sensitive systems, but may cost more to operate.

Hybrid cloud	Combines private/on-premise infrastructure with public cloud.	Best overall fit: sensitive records can have tighter controls while public cloud handles variable demand.
Community cloud	Infrastructure is shared by organisations with common requirements.	Could support a group of universities or affiliated colleges with common governance.

1.4 Current Problems and Cloud Solutions

Current problem	**Related cloud characteristic**	**How it helps**
12 servers average only 15% CPU use	Resource pooling and measured service	Consolidates workloads and reduces idle hardware and power costs.
CPU peaks exceed 95% during admissions and exams	Rapid elasticity	Adds resources during peak periods and scales down later.
Procurement takes 6–8 weeks and there is no DR site	On-demand self-service and broad network access	Resources can be provisioned quickly and backups/replicas can be stored in another region.

1.5 Benefits and Limitations

Benefits include lower initial capital expenditure, faster provisioning, flexible capacity, improved access across four campuses, managed backups and easier disaster recovery. Cloud services also reduce the university's need to maintain physical servers.

Limitations include recurring usage costs, dependence on internet connectivity, vendor lock-in, data privacy and compliance responsibilities, possible provider outages and the need for trained cloud administrators. Cloud migration does not automatically make an application secure or reliable; configuration and governance remain the university's responsibility.

Conclusion: A hybrid-cloud deployment is recommended because it balances control over sensitive student information with the scalability of public-cloud services.

TASK 2 – EVOLUTION OF COMPUTING AND HARDWARE

2.1 Computing Infrastructure Timeline

Stage	Main features	Limitation addressed
Mainframes	One powerful central computer served many users through terminals.	Centralised processing replaced separate manual calculations, but systems were expensive and centralised.
Minicomputers	Smaller and less expensive multi-user computers.	Made computing available to departments and medium-sized organisations.
Personal computers	Computing moved to individual desktops.	Improved personal access, but data and applications became distributed.
Client–server	Clients request services from central servers over a network.	Enabled shared databases and central application management.
Clusters	Connected computers work together as one system.	Improved performance and availability using multiple machines.
Grid computing	Distributed resources from different locations solve large tasks.	Allowed sharing of geographically distributed computing resources.
Utility computing	Computing capacity is consumed and billed like a utility.	Introduced usage-based provisioning and payment.
Cloud computing	Virtualised, network-accessible, elastic services delivered on demand.	Combines resource pooling, automation, elasticity and measured billing.

2.2 Hardware Evolution

Multi-core processors: Several processing cores execute tasks in parallel, increasing throughput.

Commodity servers: Large numbers of standard servers can be combined at lower cost instead of depending only on expensive specialised machines.

High-capacity storage: Distributed storage systems hold large student records, documents, videos and backups.

High-speed networking: Faster networks connect data-centre servers and allow users at different campuses to access services.

Modern data centres: Power, cooling, physical security, redundant network links and automated management support reliable large-scale computing.

2.3 How Each Stage Enabled Cloud

Cloud computing became practical because processors became more powerful, storage became cheaper, networks became faster and data centres became highly automated. Virtualisation then allowed several isolated workloads to share one physical server. Automation made it possible to provision and monitor resources on demand.

2.4 University's Current Stage

The university currently uses a traditional client–server, on-premise data-centre model. Each application has a dedicated physical server. Low average CPU use and high peak utilisation show poor resource utilisation and limited elasticity. It has not yet adopted a fully virtualised, elastic cloud model.

TASK 3 – EVOLUTION OF INTERNET SOFTWARE

3.1 Stages of Internet Software Evolution

Stage	Description	University example
Static web pages	Fixed HTML pages display the same content to visitors.	A public page showing university contact details.
Dynamic web	Server-side programs generate content based on user input and databases.	A student logs in and views personal attendance.
Web 2.0	Interactive websites support user participation and frequent updates.	Students submit feedback or participate in discussion forums.
Service-Oriented Architecture (SOA)	Application functions are exposed as reusable services with defined interfaces.	A fee service can be used by the student portal and mobile app.
Mashups	Information from multiple services is combined into one interface.	A campus dashboard combines maps, timetable and weather information.
Cloud-hosted applications	Applications run on scalable cloud infrastructure and are accessed through the internet.	A cloud-hosted LMS serves students across all campuses.

3.2 Role of Browser, HTTP, AJAX and APIs

Browser: Displays the user interface and sends requests to web servers. Example: a student uses Chrome to open the exam portal.

HTTP/HTTPS: Transfers requests and responses between clients and servers. HTTPS encrypts data in transit using TLS.

AJAX: Allows a web page to exchange data with a server without reloading the entire page. Example: displaying search results while a student types.

API: Defines how applications communicate. Example: the mobile app calls a result API and receives JSON data.

3.3 Desktop, Client–Server and Web Software

Type	Processing and access	Advantages	Limitations
Desktop software	Installed and mainly runs on one computer.	Can work offline; direct access to local resources.	Updates must be installed on devices; sharing data is harder.
Client–server software	Client application communicates with a central server.	Central data and shared services.	Client installation and server capacity must be managed.
Web-delivered software	Browser accesses an application hosted on a server or cloud.	No complex client installation; central updates; access across devices.	Needs network access and careful browser/API security.

3.4 Effect on University Application Design

The university should gradually separate its monolithic applications into well-defined modules and APIs. The student portal, mobile app and partner-college systems should use the same authenticated services rather than maintaining separate copies of student and fee data. This reduces duplication and makes future changes easier.

TASK 4 – SERVER VIRTUALIZATION

4.1 Meaning and Working of a Hypervisor

Server virtualisation divides the computing capacity of one physical server into multiple isolated virtual machines (VMs). A hypervisor is the software layer that creates and manages VMs. It allocates CPU, memory, storage and network resources to each VM.

4.2 Type-1 and Type-2 Hypervisors

Type	Working	Examples	Use
Type-1 (bare-metal)	Runs directly on physical server hardware.	VMware ESXi, Microsoft Hyper-V, Xen.	Recommended for production university servers because it has lower overhead and central management.
Type-2 (hosted)	Runs as an application on an existing operating system.	Oracle VirtualBox, VMware Workstation.	Useful for student labs, demonstrations and local testing.

4.3 Virtualisation Techniques

Technique	Explanation	Example / use
Full virtualisation	Guest operating system runs in a virtual hardware environment without needing modification.	Run a legacy Linux or Windows application in a VM.

Para-virtualisation	Guest OS is aware of the hypervisor and uses special interfaces for efficient operations.	Useful where guest and hypervisor support para-virtualised drivers.
OS-level virtualisation	Containers share the host operating-system kernel but isolate applications and dependencies.	Docker containers for stateless web/API services.
Hardware-assisted virtualisation	CPU features such as Intel VT-x or AMD-V help the hypervisor run guest operating systems efficiently.	Modern production VM hosts.

4.4 VM and Container Placement

Use VMs for legacy monolithic applications that require their own operating system, such as the existing SIS, examination system and older library software. Use containers for new stateless web services, API components and independently deployable modules. Databases may run on managed database services where feasible, or on dedicated VMs if compatibility or control requires it.

4.5 Server Consolidation Calculation

Given: 12 physical servers, each operating at an average CPU utilisation of 15%. Assume all servers have equal CPU capacity and workloads can be consolidated without a performance penalty. Target host utilisation is 70%.

Total current average CPU load = 12 × 15% = 180% of one server's CPU capacity.

Required hosts = Total load ÷ Target utilisation = 180% ÷ 70% = 2.57.

Therefore, the mathematical minimum is 3 hosts. Check: 3 × 70% = 210% aggregate usable target capacity, which is greater than the estimated 180% load.

Practical recommendation: deploy at least 4 hosts initially, not just 3, to provide N+1 capacity during maintenance or a host failure. This estimate uses average CPU only. Actual sizing must also check RAM, storage IOPS, network throughput and peak demand. A 15% average does not prove that three hosts can handle the admissions/exam peak.

4.6 Benefits and Risks

Live migration: Moves a running VM between compatible hosts with little interruption, useful during maintenance.

Snapshots: Capture a VM's disk state before a change, helping rollback. Snapshots are not a replacement for independent backups.

Resource over-commitment: Assigns more virtual CPU or memory than physically available based on expected usage. Excessive over-commitment can cause contention.

Noisy neighbour: One VM can consume excessive shared resources and slow other VMs. Use quotas, monitoring and workload isolation.

Hypervisor failure: A host or hypervisor failure can affect several VMs. Use multiple hosts, clustering and tested recovery procedures.

Security isolation: A hypervisor vulnerability or incorrect configuration may expose workloads. Patch hosts, restrict management access and segment networks.

TASK 5 – WEB SERVICES

5.1 Meaning and Role

A web service is a software interface that allows two applications to exchange data over a network using standard protocols. In a Service-Oriented Architecture, business functions are exposed as reusable services. The university can therefore share approved functions with its mobile app and 40 affiliated colleges without giving those systems direct database access.

5.2 SOAP, WSDL, UDDI and REST

Technology	Meaning	Typical use
SOAP	Protocol using structured XML messages and defined rules; can work with enterprise security and transaction standards.	Complex enterprise integrations requiring strict message contracts.
WSDL	XML document describing a SOAP service, its operations and message formats.	Documents how a client calls a SOAP service.
UDDI	Directory standard for publishing and discovering web services.	Historically used to locate service descriptions.
REST	Architectural style commonly using HTTP methods and resource URLs; often exchanges JSON, but XML is also possible.	Mobile apps, portals and partner integrations.

5.3 Proposed University APIs

Function	Endpoint and method	Sample request	Sample response	Recommended style
Result lookup	GET /api/v1/students/{studentId}/results?semester=4	Authenticated request for studentId=STU1024, semester=4	{"studentId":"STU1024","semester":4,"result":"PASS","gpa":8.2}	REST: simple read-only resource lookup; JSON is mobile-friendly.

Fee payment	POST /api/v1/payments	{"studentId":"STU1024","amount":25000,"orderId":"ORD901"}	{"paymentId":"PAY501","status":"PENDING","orderId":"ORD901"}	REST over HTTPS with payment gateway integration, idempotency key and signed callback. SOAP may be chosen if a bank mandates it.
Library search	GET /api/v1/library/books?query=cloud	Search term query=cloud	{"items":[{"bookId":"B101","title":"Cloud Computing"}],"count":1}	REST: search is naturally represented as a GET request.

The examples are illustrative API contracts, not live university endpoints. The university must validate the caller's identity and permission on every request. Student records should never be exposed using an unprotected public endpoint.

5.4 Security and API Design

Use HTTPS for all API traffic.

Use OAuth 2.0 or OpenID Connect for user and application authentication.

Apply role-based access control and tenant-specific permissions.

Validate input, rate-limit requests and log security-relevant events.

Use versioned endpoints such as /api/v1 so changes do not unexpectedly break existing mobile apps.

TASK 6 – SERVICE MODELS: IAAS, PAAS AND SAAS

6.1 Definitions and Responsibility

IaaS (Infrastructure as a Service) provides virtual machines, storage and networking. The provider manages physical infrastructure and virtualisation; the university manages the operating system, middleware, application and data.

PaaS (Platform as a Service) provides a managed platform for deploying applications. The provider manages infrastructure, operating system and runtime; the university manages its application code and data.

SaaS (Software as a Service) provides a complete application through the internet. The provider manages the application and underlying platform; the university manages users, access settings and its use of the data.

Layer	IaaS	PaaS	SaaS
Physical data centre / hardware	Provider	Provider	Provider
Virtualisation / operating system	Provider manages infrastructure; customer manages guest OS	Provider	Provider
Runtime / middleware	Customer	Provider	Provider
Application	Customer	Customer	Provider
Data and user access	Customer	Customer	Shared responsibility; customer configures users and governs its data

6.2 University Application Mapping

Application	Suggested model	Reason
Student Information System (SIS)	IaaS initially, then modernise toward PaaS	Legacy monolithic SIS may require OS-level control during migration. New modules can move to managed platforms.
Learning Management System (LMS)	SaaS	A mature hosted LMS reduces infrastructure maintenance and can scale for online learning.
Library System	SaaS if a suitable product meets requirements; otherwise IaaS	Library workflows are common and can be provided by specialist software. Legacy integrations may require temporary IaaS.
Exam Portal	PaaS with containers or managed application services	The university controls exam logic while the platform provides deployment, scaling and managed runtime. Strong testing and security are essential.
Email	SaaS	Hosted email provides mail, calendar, spam filtering and maintenance without operating university mail servers.

6.3 Model Comparison

Factor	IaaS	PaaS	SaaS
Control	Highest control over OS and software stack.	Control over application code; less control over runtime.	Least technical control; configure product features and users.

Cost	Can be cost-effective but requires OS, patching and operations staff.	Reduces platform administration; usage and platform costs apply.	Usually subscription-based per user or usage.
Scalability	Can scale, but VM capacity and configuration must be managed.	Often supports built-in scaling and managed deployment.	Provider scales the product, within plan limits.
Maintenance	High customer responsibility.	Shared; provider maintains platform.	Provider maintains application; customer governs configuration and data.
Vendor lock-in	Moderate; VM images and services may be portable with effort.	Can be high due to proprietary runtimes and services.	Can be high due to data formats, workflows and integrations.

Provider examples: AWS EC2, Azure Virtual Machines and Google Compute Engine are IaaS examples. Google App Engine, Azure App Service and AWS Elastic Beanstalk are PaaS examples. Microsoft 365, Google Workspace and a hosted LMS product are SaaS examples. The university should select providers through security, compliance, support and total-cost evaluation, not brand popularity.

TASK 7 – XAAS (ANYTHING AS A SERVICE)

7.1 Meaning and Variants

XaaS means Anything-as-a-Service. It is a broad term for delivering technology capabilities as services over a network, usually through subscription or usage-based pricing.

DBaaS (Database as a Service): A managed database where the provider handles backups, patching and much of the database infrastructure.

DRaaS (Disaster Recovery as a Service): Replication and recovery capabilities that help restore systems after a major outage.

FaaS (Function as a Service): Runs short pieces of code in response to events without the university managing servers.

BaaS / MBaaS (Backend as a Service / Mobile Backend as a Service): Provides common mobile backend functions such as authentication, notifications and data services.

DaaS (Desktop as a Service): Delivers virtual desktops to users over a network.

Security-as-a-Service: Provides managed security capabilities such as threat detection, email protection or identity monitoring.

7.2 Two Recommended Offerings

1. DBaaS: A managed relational database can provide automated backups, patching, monitoring and high availability for student and fee records. It reduces database administration effort. The university must still configure access, encryption, retention and recovery tests.

2. DRaaS: A separate recovery environment helps restore critical services if the primary cloud region or data centre becomes unavailable. Recovery time objective (RTO) and recovery point objective (RPO) must be defined and tested; merely buying DRaaS does not prove recovery will work.

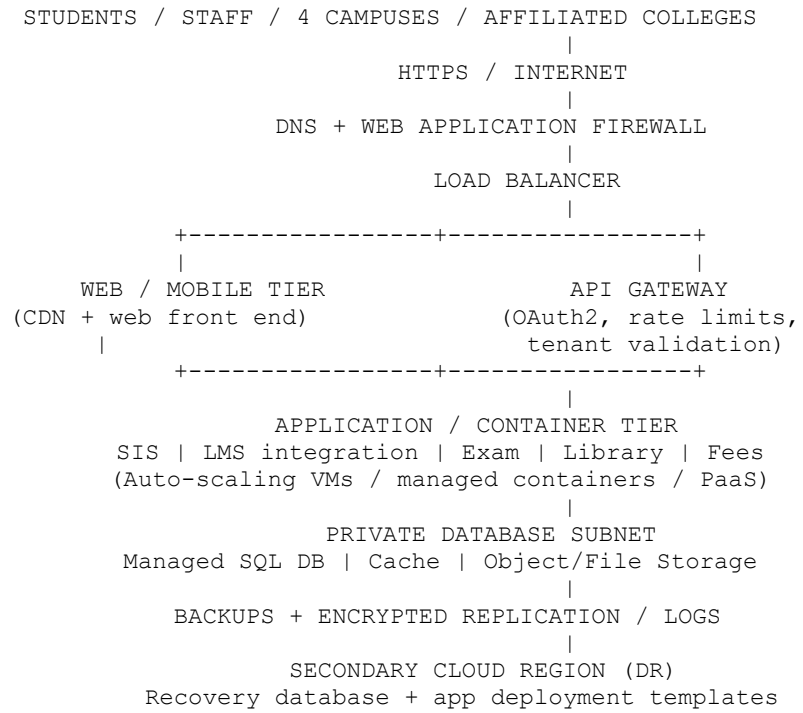
7.3 Risks of Multiple Providers

Using many providers can create fragmented identity management, inconsistent security settings, data-transfer costs, complex support arrangements, integration failures and vendor lock-in. The university should maintain a provider inventory, standard APIs, central identity management, documented exit plans and regular recovery tests.

TASK 8 – CLOUD SOLUTION ARCHITECTURE

8.1 Proposed Architecture

The recommended design is a hybrid deployment. Public-cloud services host scalable web, API and application components across multiple availability zones. Sensitive data is protected with encryption, strict identity controls and a managed database configuration that meets the university's legal and institutional requirements. A separate recovery region stores backups and selected replicas.



8.2 Component and Service Model Choices

Component	Model / deployment	Reason
DNS, CDN and web application firewall	Public cloud managed service	Improves delivery speed and filters common web attacks.
Load balancer	IaaS/PaaS managed service	Distributes traffic across healthy application instances.
Web and API tier	PaaS / containers	Supports independent deployment, scaling and API security.
Legacy SIS components	IaaS VMs during transition	Maintains compatibility while the application is modernised.
New application modules	PaaS / containers	Reduces operating-system administration and supports faster releases.
Database	DBaaS in private network	Managed backups and maintenance with restricted network access.
Documents and backups	Object storage / XaaS	Scalable storage with lifecycle and retention policies.
Disaster recovery	DRaaS / secondary region	Provides recovery capability outside the primary region.

8.3 Elasticity, Security and Availability

Use auto-scaling based on CPU, memory, request rate and queue length. During admissions and examinations, additional instances are launched; after demand falls, instances are removed. Capacity limits and budget alerts prevent uncontrolled spending.

Deploy application instances across at least two availability zones.

Use multi-factor authentication for administrators and role-based access for students, staff and partner colleges.

Encrypt traffic with HTTPS/TLS and encrypt databases, object storage and backups at rest.

Keep databases in private subnets; do not expose database ports directly to the internet.

Use automated backups, point-in-time recovery where supported, and periodically test restoration.

Monitor application health, error rates, latency, resource use and security events.

Maintain a secondary-region recovery plan and document RTO and RPO.

A 99.9% availability target permits approximately 0.1% downtime: 8.76 hours in a 365-day year. This target must be measured end-to-end and supported by service-level agreements, monitoring and tested failover.

TASK 9 – HANDS-ON IMPLEMENTATION AND DOCUMENTATION

This task requires actual execution evidence. The following is a reproducible local proof-of-concept using Flask and Docker. The commands and expected responses are provided as a guide; they are not proof that the code has been executed on the student's computer. Add genuine screenshots and actual command output after running it.

9.1 Create the Flask REST API

Create a folder named university-api. Inside it, create app.py with the following code:

```
from flask import Flask, jsonify

app = Flask(__name__)

STUDENTS = {
    "STU1024": {
        "student_id": "STU1024",
        "name": "Sample Student",
        "department": "CSE",
        "semester": 4,
        "result": "PASS"
    }
}

@app.get("/")
def home():
    return jsonify({"message": "University API is running"})

@app.get("/api/v1/students/<student_id>")
def get_student(student_id):
    student = STUDENTS.get(student_id)
    if student is None:
        return jsonify({"error": "Student not found"}), 404
    return jsonify(student), 200

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

Create requirements.txt:

```
Flask==3.1.0
```

Create a Dockerfile:

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
EXPOSE 5000
CMD ["python", "app.py"]
```

9.2 Build and Run

```
docker build -t university-api .
```

```
docker run --name university-api-container -p 5000:5000 university-api
```

Open a browser at http://localhost:5000/ or test with curl. In a second terminal, run:

```
curl http://localhost:5000/api/v1/students/STU1024
```

Expected successful response (the exact formatting may differ):

```
{"department":"CSE","name":"Sample
Student","result":"PASS","semester":4,"student_id":"STU1024"}
```

Failure test: run curl http://localhost:5000/api/v1/students/UNKNOWN. Expected HTTP 404 and {"error":"Student not found"}.

9.3 PaaS / SaaS Component

For a simple PaaS demonstration, deploy the same Flask application to a managed application platform such as Azure App Service, Google App Engine or another approved free-tier service. Follow the provider's current account, billing and deployment requirements. Free-tier limits and availability change, so check the provider's current terms before deployment.

9.4 Evidence to Attach

Screenshot 1: project files (app.py, requirements.txt and Dockerfile).

Screenshot 2: successful Docker image build.

Screenshot 3: running container shown by docker ps.

Screenshot 4: browser or curl output for the successful student lookup.

Screenshot 5: HTTP 404 response for an unknown student.

Screenshot 6: PaaS deployment dashboard and live endpoint, if deployment is completed.

Do not submit expected output as if it were captured output. Run the commands, capture your own screenshots, and record any errors and fixes.

TASK 10 – EVALUATION AND FINAL RECOMMENDATION

10.1 Comparison of Three Options

Factor	Existing on-premise	IaaS-based cloud	PaaS/SaaS-based cloud
Cost pattern	High capital expenditure for servers, power, cooling and replacement.	Lower initial CapEx but ongoing VM, storage, network and support charges.	Mostly operating expenditure; managed services reduce administration but subscriptions and usage charges continue.
Scalability	Limited by installed hardware and slow procurement.	Can add VMs, but configuration and capacity management remain necessary.	Managed scaling is generally easier for supported workloads.
Availability	Single data centre and no DR site are major weaknesses.	Can use multiple zones and backup regions if designed correctly.	Provider operates the service, but the university must check SLA and configuration.
Security	Direct control, but patching and resilience depend on local capability.	Shared responsibility; university manages guest OS and applications.	Provider manages more layers; university still controls identities, data and settings.

Effort	High hardware and infrastructure maintenance.	Medium to high system administration effort.	Lower platform maintenance, but integration and vendor management remain.

10.2 Illustrative Three-Year Cost Estimate

The following is a planning estimate in Indian rupees, not a vendor quotation. Assumptions: 12 legacy servers are replaced or consolidated; costs include infrastructure, storage, backup, networking and estimated support/operations. Actual pricing depends on provider, region, workload, discounts, taxes, licensing and traffic.

Option	Assumed annual cost	Three-year total
On-premise refresh	₹30,00,000	₹90,00,000
IaaS cloud	₹24,00,000	₹72,00,000
PaaS/SaaS-led hybrid	₹20,00,000	₹60,00,000

Illustrative calculation: On-premise = ₹30 lakh × 3 = ₹90 lakh. IaaS = ₹24 lakh × 3 = ₹72 lakh. PaaS/SaaS-led hybrid = ₹20 lakh × 3 = ₹60 lakh. Under these assumptions, the PaaS/SaaS-led option is ₹30 lakh lower over three years than the on-premise refresh. These figures are not guaranteed savings. A proper decision requires a workload inventory, provider price calculator, data-egress estimate, licence review and migration-cost estimate.

10.3 Risks and Mitigation

Risk	Mitigation
Vendor lock-in	Use documented APIs, standard data formats, infrastructure-as-code and an exit/migration plan.
Data privacy or unauthorised access	Encrypt data, apply least privilege, MFA, audit logs, retention controls and periodic access reviews.
Cloud or network downtime	Use multiple availability zones, resilient connectivity, backups and tested disaster recovery.
Skills gap	Train administrators, document operating procedures and use a phased migration with expert support where needed.
Unexpected cost growth	Set budgets and alerts, tag resources, review usage monthly and remove unused resources.
Migration failure or data loss	Run pilot migrations, validate data, maintain backups and define rollback criteria.

10.4 Migration Roadmap

Phase 1 – Assess: Inventory applications, dependencies, data sensitivity, peak loads, licences and recovery needs.

Phase 2 – Foundation: Establish cloud accounts, network connectivity, identity management, security policies, monitoring and backup.

Phase 3 – Low-risk migration: Move email or a suitable collaboration service to SaaS and migrate static public content.

Phase 4 – Pilot: Containerise a small API or library integration and test performance, security and cost.

Phase 5 – Core systems: Migrate SIS and examination components in stages, beginning with non-critical modules and maintaining rollback options.

Phase 6 – Optimise: Tune auto-scaling, database performance, cost controls, disaster recovery and service-level monitoring.

Final recommendation: adopt a hybrid-cloud strategy with SaaS for email and suitable LMS/library functions, PaaS/containers for new APIs and web applications, IaaS for legacy systems during transition, and DBaaS/DRaaS where security and recovery requirements are met. This approach avoids a risky all-at-once migration and reduces the university's dependence on under-utilised physical servers.

TASK 11 – DAY-4 DESIGN CHALLENGE: RE-EVALUATE AND ADAPT

11.1 Impact Analysis

The new requirement changes the system from a campus-focused service into a wider digital platform. It must support a mobile app, 40 affiliated colleges, student-record access and online fee payments. The expected load is ten times the previous peak, availability must be 99.9%, and the budget remains limited.

User population and access patterns increase because external college users join the four campuses.

Student and fee APIs become critical shared interfaces for mobile apps and partner systems.

Multi-tenant access introduces the risk that one college may view or modify another college's records.

Payment processing requires secure integration, transaction traceability and protection against duplicate payment requests.

11.2 Evaluation of the Original Design

A basic VM-based design with one application tier and one region is not sufficient evidence of readiness for a ten-times peak. Auto-scaling helps only when the application is stateless or its state

is stored in shared services, and when the database, network and third-party payment provider can handle the increased load.

A single-region deployment may have good local redundancy but remains exposed to a regional outage. To target 99.9%, deploy across multiple availability zones and prepare a secondary-region recovery plan. The 99.9% target allows about 8.76 hours of downtime per year, but actual service availability must include dependencies such as DNS, identity, database and payment services.

11.3 Adaptation / Revised Architecture

Decision: modify the original design rather than retain it unchanged or rebuild everything. Keep the hybrid-cloud direction, but strengthen the API, application, database, tenancy and recovery layers.

Place an API gateway in front of all partner and mobile APIs. It performs authentication, rate limiting, request validation, API versioning and logging.

Use container orchestration or managed application services for stateless API components. Scale replicas horizontally across multiple availability zones.

Use a managed relational database with connection pooling, read replicas where suitable, indexing and tested backup/restore. Keep payment writes transactionally consistent.

Use a queue for slow or bursty background work such as notifications, report generation and non-urgent synchronisation.

Use a payment gateway; do not store card details. Use idempotency keys to prevent duplicate payment processing.

Use tenant-aware authorisation. Every request must carry a verified college identity and the backend must enforce tenant_id restrictions on every data access.

Use a secondary region for backups and recovery. Define and test RTO and RPO rather than assuming that replication alone guarantees recovery.

11.4 Multi-Tenant Security

Each affiliated college should have a unique tenant identifier and its own authorised roles. The server must derive tenant identity from a trusted token or verified client registration, not from an untrusted value supplied in the request body. Database queries must enforce tenant boundaries. Administrative access should be logged and reviewed. For especially sensitive data, separate databases or schemas may be considered, but this increases cost and operational complexity.

11.5 Capacity and Availability Calculations

Let the measured original peak request rate be P requests per second. The revised peak requirement is 10P. If one tested application instance safely handles C requests per second at the required latency, the theoretical instance count is N = ceiling(10P ÷ C). For example, if the original peak is 100 requests/second and one instance safely handles 50 requests/second, the revised load is 1,000 requests/second and requires at least ceiling(1,000 ÷ 50) = 20 instances. Add headroom for failures and uneven traffic; the final value must come from load testing.

Availability calculation: 365 × 24 × 60 = 525,600 minutes per year. Allowed downtime at 99.9% = 0.001 × 525,600 = 525.6 minutes, which is 8.76 hours per year. A design using multi-zone deployment, health checks, automated failover and tested recovery is more appropriate than a single-zone design.

11.6 Cost Comparison and Trade-offs

The revised design may cost more than the original because it needs multi-zone capacity, an API gateway, stronger database capacity, monitoring and disaster recovery. Costs can be controlled by autoscaling, serverless processing for intermittent tasks, storage lifecycle rules, reserved capacity for predictable baseline demand and budget alerts. Do not promise that ten-times load will cost only ten times more or that autoscaling will always reduce costs; database and network costs may become the bottleneck.

Area	Original design	Revised design
Application	VM-centric web/application tier	Stateless containers or managed PaaS with horizontal scaling
External access	Basic web services	API gateway with partner authentication, throttling and versioning
Availability	Primary deployment with backup	Multi-zone primary region plus secondary-region recovery
Data	Central database	Managed database tuned for peak load, tenant isolation and tested recovery
Processing	Synchronous requests	Synchronous critical transactions plus queues/serverless background tasks
Security	User access controls	Tenant-aware authorisation, per-college isolation, MFA and detailed audit logs

Trade-offs: the revised architecture improves elasticity, availability and partner integration, but adds operational complexity, more cloud services and possible vendor lock-in. The university should standardise interfaces, keep infrastructure definitions documented, test failover and compare provider costs before committing.

11.7 Final Decision

The original hybrid-cloud recommendation should be modified. Retain the hybrid approach, but introduce an API gateway, tenant-aware security, multi-zone application deployment, scalable managed data services, queue-based background processing and a secondary-region recovery plan. Before production launch, conduct load tests at ten times the measured peak, security tests for cross-tenant access, payment failure tests and disaster-recovery drills. If these tests fail, the system is not ready for the Day-4 requirement.

OVERALL CONCLUSION

Cloud computing can solve the university's low server utilisation, slow procurement and lack of disaster recovery, but migration must be planned around application dependencies, security, peak demand and cost. A hybrid-cloud strategy supported by virtualisation, APIs, appropriate IaaS/PaaS/SaaS choices and selected XaaS services is the most balanced solution. The Day-4 requirement makes API governance, multi-tenant isolation, multi-zone availability and measured load testing essential. The hands-on task must be completed with genuine execution screenshots before final submission.